

Unveiling Cognitive Attacks on Multimodal LLM-based XR Cognitive Assistants through Trojans

Anonymous Author(s)

Abstract

Extended Reality (XR), empowered by large vision-language models (LVLMs), rapidly transforms user interactions through complex tasks by overlaying visual cues and textual instructions, enhancing performance in daily and high-stakes scenarios. However, this growing reliance also introduces new, underexplored security risks, particularly those that subtly target users' perception and cognition rather than system infrastructure. In this paper, we introduce, for the first time, a cognitive attack Trojan that targets LVLM-guided intelligent XR cognitive assistants by subtly manipulating systems' perception to degrade user task performance. Unlike other attacks, this Trojan remains hidden within the application and does not interfere with the application's functionality until the attack is triggered. Once triggered, such attacks increase users' cognitive load, extend task completion times, and disrupt the immersive flow of the XR experience, leading to frustration and reduced trust in the intelligent XR cognitive assistant. To demonstrate the effectiveness of this attack, we first develop *CogXR*, an LVLM-guided intelligent XR cognitive assistant on the Magic Leap 2 headset that provides real-time visual and textual guidance for diverse cognitive tasks. We then evaluate *CogXR* across two distinct cognitive task categories: spatial reasoning tasks (e.g., Rubik's Cube solving) and procedural tasks (e.g., cooking) to assess how the attack influences performance under varying levels of cognitive demand. Our findings demonstrate that, when triggered, the Trojan subtly manipulates visual input, misleading the LVLM across both task categories, resulting in prolonged task completion times, increased cognitive demand, and disrupted immersive flow. **Our conducted user studies on both spatial reasoning and procedural tasks show that the cognitive attack Trojan increases task completion time by up to 43% on the spatial-reasoning task and 25.2% on the procedural task relative to the no-attack condition**, elevating cognitive load and degrading performance, highlighting critical vulnerabilities in XR systems.

CCS Concepts

- **Security and privacy** → **Software and application security**;
- **Human-centered computing** → *Mixed / augmented reality*; • **Computing methodologies** → Artificial intelligence.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
VRST '26,

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/XX
<https://doi.org/XXXXXXX.XXXXXXX>

ACM Reference Format:

Anonymous Author(s). 2026. Unveiling Cognitive Attacks on Multimodal LLM-based XR Cognitive Assistants through Trojans. In *Proceedings of ACM Symposium on Virtual Reality Software and Technology (VRST '26)*. ACM, New York, NY, USA, 13 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

The recent trend of integrating advanced pre-trained large vision-language models (LVLMs) into extended reality (XR) systems is revolutionizing their capabilities, enabling personalized, context-aware interactions that significantly reshape our engagement with the world. One such notable application is LVLM-driven intelligent XR systems, denoted as intelligent XR cognitive assistance, which overlay step-by-step instructions onto users' fields of view and guide them through complex tasks that require spatial reasoning and systematic thinking. **For instance, spatial reasoning tasks (e.g., puzzle and configuration solving), where visual understanding and methodical reasoning are essential, or procedural tasks (e.g., assembly, maintenance, and cooking), which require sequential step-by-step guidance, are significantly enhanced by LVLMs in XR environments** [22, 23, 35, 50, 51, 55, 61, 67, 69, 77].

Furthermore, researchers have also integrated LVLMs into XR systems to facilitate nuanced tasks such as pronoun disambiguation for voice assistants [36], intelligent XR-based activity recording and analysis [28], documentation and writing support [12], augmented object intelligence [21], understanding user behaviors and contextual reasoning [30], and even harmful content detection within XR environments. These advancements underscore the role of LVLMs in transforming XR into a robust cognitive augmentation platform that supports users in both mundane and mission-critical scenarios. Beyond this, such integrations are also expected in *safety-critical applications*, such as the recently announced next-generation military XR headsets, designed and built by Anduril and Meta to provide warfighters with enhanced perception, assistance, and enable intuitive control of autonomous platforms on the battlefield [5].

As LVLMs increasingly permeate daily life, they also pose significant threats to users' safety, security, and privacy [3, 17, 32]. *However, since leveraging LVLM to XR is still a new concept, the vulnerabilities of such integrations are yet vastly underexplored.* For instance, a slight alteration in an object's appearance or insertion of an adversarial element can cause the LVLM to misidentify critical features or misunderstand user intent [70, 72, 73, 75]. Unfortunately, as research on the typical security and privacy consequences of XR and LVLM integration grows, the *cognitive security* [19, 20] of this integration for intelligent XR cognitive assistants has received little attention from the community. Indeed, cognitive security is critical across many applications, as the state of the user's cognition is directly related to stress, task performance, decision-making, and other factors crucial to applications such as public safety and national defense. For instance, in a battlefield situation, the compromise of a soldier's cognitive security can lead to catastrophic

consequences [19]. Commercial XR systems apply cognitive engineering principles during system development; however, due to the product supply chain, it is not always possible to ensure that intelligent XR cognitive assistants operate safely when facing an adversary intent on interfering with a mission. This is especially true if the threat is dormant, e.g., a *Trojan*, which can only be activated externally by a trigger (e.g., a visual trigger). Since Trojans are by definition dormant, stealthy, and stay inactive during normal operation, it is extremely challenging to detect Trojans in AI (TrojAI) [54] during the software or hardware testing phases. Hence, there is a pent-up need to explore the impact of cognitive security vulnerabilities of intelligent XR cognitive assistants. *It is worth noting that the goal of a cognitive attack differs from traditional Trojans, which compromise users' security (e.g., by stealing passwords, credit card numbers, etc.) and may go far beyond. For instance, jeopardizing the decision-making of intelligent LVLM XR assistants by triggering a Trojan during firefighting, search-and-rescue, or battlefield operations can significantly impair the cognitive capabilities of first responders and soldiers, putting human lives at stake.*

This paper introduces a novel cognitive attack Trojan (CAT) targeting LVLM-guided intelligent XR cognitive assistants. Inspired by TrojAI [54] and hardware Trojan attacks [8, 24, 31], which have long been a well-known threat in the AI and hardware security domains, these Trojans remain dormant until a specific external trigger condition is met. Once activated, they execute hidden malicious behaviors with subtle, hard-to-detect consequences, often severely affecting the system's functionality and security. Similarly, our proposed CAT remains dormant and undetectable during normal XR system operation, causing no disruption to the user experience. However, when a predefined condition is met, the Trojan becomes active (triggered) and manipulates the intelligent XR system's behavior in a manner imperceptible to the user. This results in significantly increased task completion times, elevated user cognitive load, and disruption of the immersive flow of the XR experience. To elucidate our proposed idea, we first develop a real-time **baseline** LVLM-guided intelligent cognitive assistant framework **CogXR** and implement it on the Magic Leap 2 XR headset to provide step-by-step guidance for representative spatial reasoning tasks (i.e., Rubik's Cube solving) and procedural tasks (i.e., cooking), as shown in Figure 2. CogXR processes captured frames from the XR camera using an LVLM to provide step-by-step guidance through visual overlays and concise textual instructions. We show that CogXR works as expected in an ideal environment (no attack launched), which is referred to as the benign environment and serves as the baseline in the paper. Next, to demonstrate the effectiveness of the proposed CAT, we develop a malicious version of the CogXR for the same tasks and perform a user study using the benign and malicious CogXR to analyze key performance metrics, including task completion time and cognitive load, as measured by the NASA Task Load Index (TLX) [26]. Our findings reveal that the proposed cognitive attack leads to a substantial increase in task completion time, with an average 43% increase in Rubik's Cube solving time and a 25.2% increase in cooking task time compared to benign conditions, thereby elevating cognitive load and degrading task performance. *To the best of our knowledge, this is the first work to propose cognitive attacks against the LVLM-guided intelligent XR cognitive assistant. We believe our work will help XR developers better understand the inherent vulnerabilities of*

such systems and pave the way for adopting appropriate measures to mitigate them.

2 Related Works

LVLM for cognitive tasks in XR: Recent advancements in pre-trained LVLM models (e.g., GPT-4o [2], Google Gemini [64], etc.) have enhanced multimodal understanding of visual and linguistic information. With zero-shot inference capabilities, LVLMs naturally integrate into XR applications, enhancing user experiences in immersive environments. Prior research demonstrates the effectiveness of LVLMs in providing real-time, context-aware guidance for complex tasks in XR environments across both spatial reasoning (e.g., Rubik's Cube and Sudoku solving) [22, 25, 51, 55] and procedural tasks (e.g., cooking, furniture assembly, etc.) [10, 23, 35, 41, 67]. A notable application is LVLM-guided intelligent XR cognitive assistance, which overlays step-by-step instructions onto users' fields of view to guide them through complex tasks [50, 61, 77]. For instance, Srinidhi et al. [61] demonstrated that integrating LVLMs with physical-world perception enables context-aware task guidance (e.g., coffee-making, furniture assembly). Similarly, an automatic AR instruction generation system used LVLMs for spatially grounded guidance. Such systems are now deployed at scale, exemplified by Meta's Ray-Ban Smart Glasses [68] using multimodal perception to provide real-time contextual insights. However, as LVLMs become increasingly embedded in XR cognitive assistants for high-stakes tasks, they introduce critical security risks. Existing research has identified several attack vectors targeting LVLMs-guided intelligent XR cognitive assistance. For instance, Zhang et al. [75] demonstrated attacks through environment manipulation, visual overlays, and prompt modification. However, such attacks are fundamentally limited, as they rely on easily-detectable physical perturbations and face barriers in restricted API environments [7, 27, 39], making them ineffective against real-world LVLMs-guided intelligent XR systems. In contrast, semantic-level attacks that degrade LVLM visual-semantic reasoning without leaving visible traces remain unexplored. Such attacks exploit the LVLM's internal reasoning rather than relying on detectable physical changes, allowing attackers to modify task guidance imperceptibly. This could potentially cause users to make critical errors in consequential XR cognitive tasks (e.g., cooking). This vulnerability motivates a new kind of stealthy and impactful attack that manipulates LVLM perception to degrade user cognitive load and task performance in XR cognitive assistants.

Safety and Security Risks in XR: XR systems are rapidly advancing within human-computer interaction (HCI), aiming to blur the boundaries between real and virtual environments [18, 61]. With lighter, more powerful head-mounted displays (e.g., Microsoft HoloLens [75], Magic Leap [61]), researchers are moving toward intelligent, context-aware XR that adapts to user tasks. However, this growing integration has also expanded XR's attack surface. Prior work reveals diverse security and privacy threats, including PMA, memory tampering, keystroke inference, side-channel exploits, human-joystick control, and overlay attacks [3, 4, 9, 13–15, 32, 42, 44, 47, 56, 58, 59, 62, 65, 66, 73], these attacks disrupt immersion and degrade users' cognitive state [20]. For instance, Tseng et al. [66] demonstrated that multisensory manipulation can

mislead decisions and cause physical harm, while Cheng et al. [15] showed delayed reactions under vision- and audio-based attacks. Similarly, sensor spoofing [14], GPU disruptions [44], and UI vulnerabilities [59] compromise XR reliability. Notably, malicious XR design techniques (e.g., visual obfuscation, misleading interactions) deceive users [66], and combining obstruction with misinformation prevents critical object perception [73]. Recently, Xiu et al. [70] introduced AR-VIM to detect non-occlusive visual manipulation. To counter these threats, researchers have explored LVLMs for detecting critical visual cues [27, 72]. However, these models remain vulnerable to adversarial manipulation, leading to inaccurate task instructions. Although these models perform well at detection, they remain susceptible to adversarial manipulation. Even subtle visual alterations or injected elements can cause misidentification, leading to incorrect task instructions, degraded XR performance, and reduced system reliability.

3 Proposed Cognitive Attack Trojan

In the following sections, we present the core idea of the proposed attack, along with its threat model, attack scenario, and methodology for attack modeling and execution. Details are provided below.

3.1 Hardware Trojan vs. Cognitive Attack Trojan

Hardware Trojan attacks are a well-established threat in the hardware security domain [8, 24, 31], where a Trojan is covertly embedded into a circuit or chip during the design or fabrication process. It remains dormant until a specific external condition, known as the **trigger** (e.g., a particular input signal or circuit state), is detected, upon which it executes a hidden malicious action, known as the **payload** (e.g., corrupting data or disabling functionality), often with severe and hard-to-detect consequences. Inspired by this trigger-payload concept, we introduce a Cognitive Attack Trojan (CAT) for LVLM-guided intelligent XR cognitive assistant systems. Rather than targeting hardware circuits, our proposed CAT targets the intelligent XR assistant’s visual processing pipeline, where the **trigger** is an *attacker-specified* unique object (e.g., a cup with a QR code, a star-shaped sticker, or specialized text). Importantly, the trigger is not activated by arbitrary objects in the environment; only the exact object with a **unique, predefined characteristic** chosen by the attacker causes activation, making the Trojan highly selective and stealthy. The **payload** consists of a small code segment that intercepts the current camera frame upon trigger activation and applies a targeted semantic-level manipulation (e.g., recoloring a Rubik’s Cube facelet, shifting an ingredient’s visual properties, etc.) before forwarding it to the LVLM, as illustrated in Figure 1.

3.2 Cognitive Attack Trojan Concept

LVLM-guided intelligent XR cognitive assistant systems rely heavily on visual perception, real-time frame capture, and accurate environment understanding. To build such systems, developers commonly integrate SOTA open-source third-party dependencies (e.g., libraries, packages, and assets), such as OpenCV for Unity [60] for real-time image processing and OpenAI for Unity [52] for LVLM API access. An attacker can exploit this trust by publishing malicious third-party dependencies on popular platforms such as

```

1 // Malicious code concealed within a third-party library
2 // Library publicly provides legitimate frame preprocessing utilities
3 public class FramePreprocessor: IFramePreprocessor{
4     private string predefinedTriggerObject = GetPredefinedTriggerObject();
5     private bool isTriggered = false;
6     // Hook intercepts frames before forwarding to LVLM
7     public Texture2D PreprocessFrame(Texture2D frame){
8         // Legitimate preprocessing (e.g., noise reduction, buffering)
9         frame = ApplyLegitimatePreprocessing(frame);
10        // Hidden Trojan logic runs silently alongside legitimate processing
11        bool triggerOK = DetectTrojanTriggerObject(frame, predefinedTriggerObject);
12        if (triggerOK && !isTriggered){
13            isTriggered = true;
14            ExecutePayload(frame, GetTargetObject());
15        }
16        return frame;
17    }
18    private bool DetectTrojanTriggerObject(Texture2D frame, string predefinedTrigger){
19        Mat frameMat = ConvertToMat(frame);
20        bool detected = ObjectDetectionPipeline(frameMat, predefinedTrigger);
21        return detected;
22    }
23    private void ExecutePayload(Texture2D frame, string targetObject){
24        ApplySemanticManipulation(frame, targetObject);
25    }
26 }

```

Figure 1: Cognitive attack Trojan trigger detection and semantic payload execution, representing malicious code embedded within a compromised third-party library that hooks into LVLM-guided intelligent XR cognitive assistant.

GitHub or the Unity Asset Store. Once integrated, any of these dependencies can covertly embed the CAT (see Section 3.1), which activates silently at runtime and semantically manipulates the camera frame (e.g., flipping digits, or changing colors) before LVLM inference, as illustrated in Figure 1. Since the component genuinely delivers its advertised functionality, developers integrate it without suspicion. Moreover, since the CAT remains dormant in the absence of the trigger object, it is extremely difficult for testers or users to detect. Furthermore, exploiting the egocentric nature of XR environments, the visual trigger can be introduced by any individual aware of it, not necessarily the original attacker, allowing multiple actors to independently activate the dormant CAT and creating a uniquely stealthy, difficult-to-detect attack vector.

3.3 Threat Model

Attacker Goals: The attacker’s primary objective is to subtly degrade user task performance within LVLM-guided intelligent XR cognitive assistant systems. These systems are designed to provide XR-based multimodal guidance across a range of real-world tasks, including procedural tasks (e.g., cooking, coffee making, industrial assembly and maintenance, and tactical procedure execution in battlefields, etc.) as well as spatial-reasoning tasks (e.g., Rubik’s Cube and Sudoku solving, hazard localization in firefighting, and route planning in search-and-rescue, etc.) [22, 23, 35, 51, 55, 57, 61, 67, 77]. By manipulating selected visual inputs, the adversary causes the LVLM to generate erroneous yet contextually plausible instructions, making tasks harder to complete while the system appears to function normally. The impact varies significantly across task contexts. For instance, in everyday tasks (e.g., cooking, Rubik’s Cube solving, furniture assembly), the attack induces incorrect actions that disrupt task flow, increase task completion time and cognitive demand, and elevate user frustration. In mission-critical operations, such as hazard localization in firefighting, route planning in search-and-rescue, and tactical procedure execution on battlefields [57], the

consequences become far more severe, as impairing the decision-making of intelligent XR assistants can critically compromise the situational awareness of firefighters, paramedics, and soldiers, directly putting human lives at stake. Across both contexts, the attack increases confusion, prolongs task completion, and elevates cognitive effort, all while remaining undetected.

Attacker Profile and Capabilities: We assume the adversary is capable of publishing a malicious library (with an embedded CAT) on widely used distribution platforms (e.g., GitHub, Unity Asset Store). The library provides genuine, useful functionality for XR development (e.g., camera frame preprocessing, LVLM API call batching, etc.), making it attractive to developers building LVLM-guided intelligent XR cognitive assistants. This attack model follows prior work that weaponized trusted software dependencies to inject malicious code into applications that rely on those libraries [34, 45]. To do so, the adversary only needs general knowledge of how egocentric frames are captured, preprocessed, and forwarded to the LVLM for reasoning [2, 50, 61, 77]. As discussed in the Attack Concept (Section 3.2), the malicious library’s legitimate frame-processing role grants the CAT natural access to the LVLM-guided intelligent XR cognitive assistant’s camera-to-LVLM visual processing pipeline, and its genuine utility ensures that the integrating developer has no reason to suspect malicious behavior. The adversary further benefits from the reality that modern XR applications rely on numerous third-party dependencies (e.g., OpenCV for Unity [60], platform SDKs) whose source code is rarely fully audited [34], making the compromised library indistinguishable from any other trusted dependencies. Leveraging this access, the adversary can design minimal yet effective manipulations (e.g., subtle color shifts, label swaps, etc.) that mislead the LVLM while remaining imperceptible to both the developer and the user. Notably, the attack can be carried out by a single actor who both develops the malicious library and later activates it, or by two independent individuals, with one embedding the CAT and the other triggering it by placing the predefined object in the camera’s view, without any coordination between them. *The beauty of this attack is that once the application is deployed with the CAT embedded, it can be launched at any time later by the adversary (or any individual aware of the attacker-specified trigger object) without accessing the headset, hardware, software, or network stack, only by placing the predefined object in the XR’s camera frame.*

3.4 Cognitive Attack Trojan Modeling and Execution

We model a CAT that leverages runtime manipulation of egocentric frames to generate incorrect instructions in LVLM-guided intelligent XR cognitive assistant systems. The XR system captures egocentric frames from the environment via a head-mounted camera and forwards them to the LVLM, which interprets the visual context and generates step-by-step multimodal guidance for task completion. These systems support a wide range of cognitively guided tasks, such as spatial reasoning tasks (e.g., Rubik’s Cube solving, Sudoku solving, etc.) and procedural tasks (e.g., cooking, making coffee, assembling furniture, etc.) [22, 23, 35, 50, 51, 55, 61, 67, 69, 77]. To model frame manipulation, we represent the semantically relevant portion of the scene as a matrix $F_k \in \mathbb{C}^{n \times m}$, where $F_k(i, j)$ denotes the semantic label or class assigned to a localized

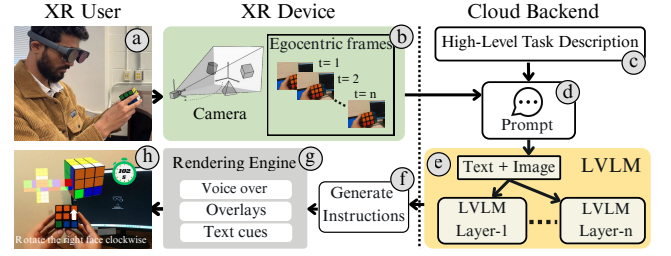


Figure 2: An overview of the *baseline CogXR* system. (a–b) An XR user performs a spatial reasoning task (Rubik’s Cube solving) while the device captures egocentric frames in real-time. (c–e) The system constructs a prompt from the captured frames and task description, and sends it to the LVLM for context-aware guidance generation. (f–h) The rendering engine delivers the generated instructions to the user via visual overlays and textual cues.

region at row i and column j . C is the set of all valid label values (e.g., object classes, colors, ingredient types, etc.) n and m refer to the number of rows and columns (spatial coordinates), and k denotes the frame index in the sequence. As discussed in the threat model, the attacker has flexibility in which element to perturb. For instance, in spatial reasoning tasks (e.g., Rubik’s Cube solving, Sudoku solving), each element in F_k represents a semantic attribute of a scene object (e.g., color, shape, position). In procedural tasks (e.g., cooking), elements in F_k may denote object properties such as categories, states, or quantities (e.g., ingredients, utensils, etc.). A single position $(i^*, j^*) \in \{0, \dots, n-1\} \times \{0, \dots, m-1\}$ is dynamically selected from the segmentation mask of the payload-specified target object, where i^* and j^* are spatial coordinates identifying the targeted semantic element in the scene. The attack matrix F_k^{att} is defined as:

$$F_k^{att}(i, j) = \begin{cases} \delta(F_k(i^*, j^*)) & \text{if } (i, j) = (i^*, j^*) \\ F_k(i, j) & \text{otherwise} \end{cases}$$

Where δ replaces the original label at $F_k(i^*, j^*)$ with a different valid label from $C \setminus \{F_k(i^*, j^*)\}$, ensuring minimal perturbation while maximizing LVLM misinterpretation.

At runtime, the system continuously detects visual entities from each frame, storing them in a set $R = \{r_1, r_2, \dots, r_n\}$, and evaluates each $r_i \in R$ against a Boolean Trojan trigger function $\phi(r_i) \in \{0, 1\}$. If no entity satisfies $\phi(r_i) = 1$, clean frames are forwarded to the LVLM unchanged. Once any $r_i \in R$ satisfies $\phi(r_i) = 1$, the Trojan activates at each subsequent decision step: it selects the most recent frame, extracts F_k , dynamically localizes the payload-specified target object, applies δ at the corresponding coordinate (i^*, j^*) , and re-embeds the modified matrix before forwarding the manipulated frame sequence F' to the LVLM $\mathcal{M}$. This perturbation persists across all subsequent decision steps as long as the trigger condition remains satisfied. Although only a single element is altered, this subtle modification distorts the model’s scene interpretation, resulting in incorrect guidance that degrades task performance, increases cognitive load, and prolongs task-completion time in LVLM-guided intelligent XR cognitive assistant systems.

Prompt 1: Example prompt for CogXR system

You are an intelligent cognitive assistant designed to support physical task execution in XR environments. From the given *<egocentric frames>* captured by the headset's front-facing camera, identify the *<key objects>* in view. Combine this information with the *<known information>* from the XR environment, the *<previous instruction>* issued to the user, the specified *<task>*, and its *<task-specific constraints>*. Analyze the user's current progress and determine the next optimal action. Provide precise, context-aware natural-language instructions grounded in the user's visual perspective to support accurate and effective task completion.

3.5 Novelty of the Proposed Attack

Our proposed CAT operates fundamentally differently from SOTA multi-modal prompt injection attacks, adversarial attacks, classical backdoor attacks on LVLMs, and XR-specific attacks (without LVLMs-guided intelligent cognitive assistants) [16, 38, 46]. For instance, multimodal prompt injection attacks manipulate LVLMs by embedding malicious instructions in text prompts (e.g., concealed text within images) or harmful visual patterns (e.g., disruptive overlays) [16, 38]. In contrast, adversarial attacks typically rely on detailed knowledge of the target model, such as its parameters or architecture, to generate pixel-level perturbations [7, 38, 74, 76]. In practice, both attack classes face barriers in commercial LVLM settings, where models are generally accessible only through restricted cloud APIs rather than as fully exposed systems [7, 46]. Similarly, classical backdoor attacks poison the training data by embedding static patterns into model weights [37, 40], requiring access during training and task-specific tuning [37]. In contrast, our CAT requires neither API access, knowledge of model internals (e.g., parameters or architectures), training data poisoning, nor modification of model weights. Instead, the Trojan activates at runtime by altering object-level properties via the attacker-specified visual trigger, remaining effective regardless of the underlying LVLM architecture. Beyond those attacks, XR-specific attacks such as perception manipulation attacks (PMA) cause broad scene-level disruption through multi-sensory manipulation [15, 66] but do not target LVLM object reasoning. Likewise, Dirksen et al. [20] address cognitive security in AR through attention-based adversarial manipulation, yet their framework operates at the scene-level attention layer rather than at the LVLM semantic reasoning level. Meanwhile, obstruction attacks overlay virtual content to occlude visibility of the real world [71], creating readily detectable visual artifacts. On the other hand, AR-VIM [70] alters text and visual patterns in AR overlays but only changes what users see, not the internal object representations LVLMs process. Similarly, Zhang et al. [75] employ query timing, environment manipulation, visual overlay injection, and prompt modification at observable input/output boundaries, making them detectable via multimodal verification [27] and LVLM safety filters [39]. In contrast, our proposed CAT modifies minimal visual elements (e.g., object color, size, or labels) before sending frames to the LVLM, targeting semantic-level object representations rather than observable channels typically examined by existing defenses. *Another distinguishing factor is that the Trojan remains dormant until triggered by any individual aware of the attacker-specified trigger object (who does not need to be the same person who implanted the Trojan), making it extremely difficult for software testers to detect. Once implanted in the application, the attack can be enabled or disabled without accessing the system.*

4 Baseline: CogXR System Design

CogXR is an intelligent system that assists users in performing complex, multi-step physical tasks by integrating the spatial perception of XR headsets with the semantic and visual reasoning of LVLMs, as shown in Figure 2. The system continuously captures frames using front-facing RGB cameras on head-mounted XR devices (e.g., the Magic Leap 2 [75]) and selectively transmits buffered frames to an LVLM backend via remote API calls or local/edge computing resources. For this, we use GPT-4o [2], which interprets captured frames, infers user progress, and generates context-aware, step-by-step instructions, leveraging its training on massive multimodal data [to produce personalized guidance across both spatial-reasoning and procedural task categories](#) [55, 61, 67]. Building on these capabilities and inspired by SOTA LVLM-guided intelligent XR cognitive assistants [50, 61, 77], we design CogXR as an intelligent, context-aware XR cognitive assistant using the structured Prompt 1, where *task* denotes the XR activity (e.g., Rubik's Cube solving, cooking), *key objects* refer to task-relevant items in the egocentric view (e.g., colors, utensils), and *task-specific constraints* capture task-dependent requirements (e.g., target cube configuration, recipe specifications). Using these structured inputs alongside egocentric frames, the LVLM analyzes user progress and generates precise next-action instructions delivered as text prompts, spatially anchored visual overlays, and voice instructions rendered in the XR interface.

Although CogXR is applicable across a wide range of tasks, as discussed in Section 3.3, we evaluate the proposed CAT on two representative categories that capture fundamentally different cognitive demands. [For spatial reasoning, we use Sudoku and Rubik's Cube solving, as both require mental rotation, geometric reasoning, and pattern recognition](#) [22, 25, 51, 55], making them well-suited for measuring performance degradation at the semantic level perturbations. [For procedural tasks, we use steak cooking and coffee making, as both involve discrete, ordered steps and continuous adaptation to the environment state](#) [10, 23, 41, 50, 61], representing practical, safety-relevant scenarios. Together, these four tasks provide complementary coverage of the cognitive demands typical of real-world XR cognitive assistants. Across all tasks, CogXR captures egocentric frames to extract the current task state into a semantic matrix representation F_k , instructs the LVLM to determine the next optimal action, and re-estimates F_k after each user action to verify execution and maintain task-state awareness, enabling progressive guidance toward completion. When the CAT is active, a single semantic perturbation to F_k (e.g., modifying a Sudoku digit, changing a Rubik's facelet color, altering a steak's color from dark to light to appear undercooked, or changing the coffee machine water level from sufficient to insufficient) corrupts the state perception, causing the LVLM to misinterpret the task configuration and generate incorrect or potentially unsafe action recommendations, as shown in Figure 3. [For task-specific visual references, please refer to the supplementary material.](#)

It is important to note that, in the remainder of this paper, we refer to the CogXR system operating in a benign environment as the baseline CogXR, and the CogXR system implanted with a CAT as the malicious version of CogXR, denoted malicious CogXR.

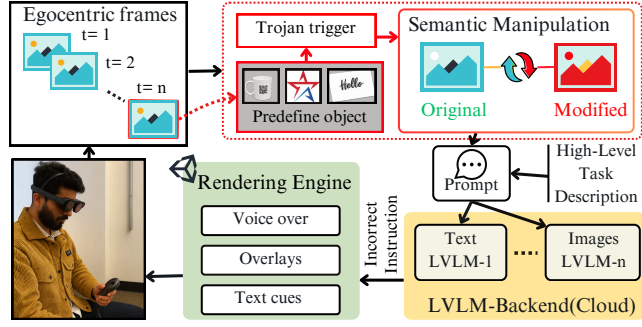


Figure 3: Overview of the CAT pipeline. XR headset frames are analyzed to detect a predefined trigger object. Upon detection, CAT alters the visual content, which makes the LVLM generate misleading instructions, which are then delivered through the XR interface.

5 System Implementation and Evaluation

We implemented both baseline CogXR and malicious CogXR in Unity 2022.3.20f1, targeting the Magic Leap 2 headset running OS version 1.12.0, using the Magic Leap Unity SDK 2.6.0. The system supports multiple XR-guided task scenarios, including spatial-reasoning tasks (Rubik’s Cube and Sudoku solving) and procedural tasks (steak cooking and coffee making). Both versions use the pre-trained GPT-4o model [2], accessed via the OpenAI API [1], as the LVLM backend to interpret egocentric visual input, estimate user progress, and generate context-aware, step-by-step multimodal guidance. For auditory feedback, we integrated Magic Leap’s text-to-speech utility to provide voice guidance during task execution. The baseline CogXR captures egocentric frames from the headset camera and uses the LVLM to deliver step-by-step guidance through visual overlays, concise text, and voice instructions. In the malicious CogXR, we simulate the attack by embedding CAT within a third-party library integrated into the application’s visual-processing pipeline, which silently alters visual input before it is forwarded to the LVLM. As shown in Figure 1, the attack comprises two synchronized phases. First, CAT continuously analyzes incoming egocentric frames to detect a predefined visual trigger (Line 11). Once the trigger is detected, CAT activates the corresponding payload (Line 14). The payload uses a locally deployed YOLO11-based instance-segmentation model [29] to dynamically localize the payload-specified target object, selects the highest-confidence instance above a predefined threshold, refines its segmentation mask using OpenCV for Unity [60], and applies the corresponding semantic modification, such as changing an object’s color or digit, before forwarding the manipulated frame to the LVLM.

System Evaluation Results: We measure end-to-end latency from headset frame capture until the generated instruction is rendered in the XR interface. As reported in Table 1, baseline CogXR requires 4.236–4.690 seconds per guidance cycle, whereas CAT-enabled CogXR requires 4.420–4.912 seconds. We independently evaluate three visual triggers, with only one trigger active per execution: a white cup with a QR code, a star-shaped sticker, and a card printed with text “Hello.” All three variants activate CAT with 100% detection accuracy. The Trigger Detection row reports

Table 1: Component-wise latency of baseline and CAT-enabled CogXR across rubik’s cub solving (RC), sudoku solving (SS), stack cooking (SC) and coffee making (CM). All values are reported in seconds. Base denotes the baseline condition, Sem. denotes semantic, and N/A denotes not applicable.

System Component	Spatial-Reasoning Tasks				Procedural Tasks			
	RC		SS		SC		CM	
	Base	CAT	Base	CAT	Base	CAT	Base	CAT
End-to-End System	4.236	4.420	4.354	4.548	4.690	4.912	4.511	4.747
GPT-4o Backend	4.169	4.175	4.285	4.292	4.612	4.625	4.438	4.449
XR Processing	0.067	0.068	0.069	0.070	0.078	0.079	0.073	0.074
Frame Capture	0.005	0.005	0.006	0.006	0.006	0.006	0.005	0.005
Response Parsing	0.041	0.042	0.041	0.042	0.047	0.048	0.045	0.046
XR Rendering	0.021	0.021	0.022	0.022	0.025	0.025	0.023	0.023
CAT Processing		0.177		0.186		0.208		0.224
Trigger Detection		0.012		0.013		0.015		0.014
QR-Coded Cup		0.012		0.013		0.015		0.014
Star Sticker		0.010		0.011		0.013		0.012
Text Card		0.014		0.015		0.017		0.016
Sem. Perturbation		0.165		0.173		0.193		0.210

the mean detection latency across the three independently tested trigger variants for each task, while the remaining CAT-column values report the corresponding mean results across these three executions. Overall, CAT introduces 0.184 to 0.236 seconds of additional end-to-end latency, corresponding to approximately 4.34 to 5.23% overhead relative to baseline CogXR.

6 Cognitive Attack Trojan Evaluation

This section presents the user study design and results, demonstrating the effectiveness of our proposed CAT on CogXR system.

Hypotheses: The primary focus of this research is to identify and understand the impact of the CAT on the CogXR system for both procedural and spatial-reasoning tasks. Specifically, we investigate whether the attack can bias the assistant’s guidance generation, producing incorrect or inefficient instructions that significantly increase task completion time, elevate cognitive demand, and disrupt the user’s immersive experience. To systematically measure and evaluate the effects of the proposed CAT on the performance and trustworthiness of the CogXR system, we formulate the following three hypotheses as the basis for our experimental study.

- **H1:** Cognitive attack Trojan will significantly increase task completion time compared to without cognitive attack Trojan condition.
- **H2:** Participants under the cognitive attack Trojan will report higher cognitive demand than those in without cognitive attack Trojan condition.
- **H3:** The timing of triggering the cognitive attack Trojan (i.e., early, mid, or late stage) will have a differential effect, with early-stage attacks causing the most significant impact on task completion time and increased cognitive demand than later-stage attacks.

Ethical Considerations. This study was approved by the authors’ Institutional Review Board (IRB). All participants provided written informed consent and brief demographic information. Privacy was protected throughout the study in compliance with IRB standards.

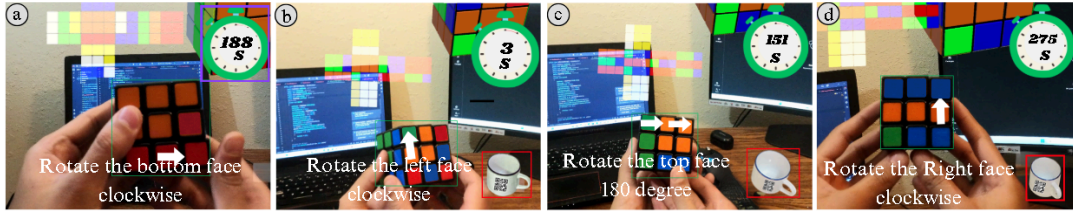


Figure 4: User's perspective during Rubik's Cube solving tasks using CogXR, where the timer is shown: (a) Baseline CogXR, (b-d) Malicious CogXR with a predefined visual trigger (e.g., a cup with a QR code) under the early-stage, mid-stage, and late-stage CAT conditions.

Table 2: Descriptive statistics of the time taken in seconds to complete rubik's cube and cooking tasks during the user study are presented for different study conditions: Baseline, Early-stage (ECAT), Mid-stage (MCAT), and Late-stage (LCAT).

Condition	Rubik's cube		Cooking	
	Mean	SD	Mean	SD
Baseline	198.5	10.31	701.2	6.98
ECAT	283.4	8.14	877.8	11.69
MCAT	251.3	11.78	850.4	7.33
LCAT	229.7	11.55	801.4	7.77

6.1 User Study: Rubik's Cube Solving

This section presents the user study design and evaluation results of our proposed CAT on CogXR for Rubik's Cube-solving task.

6.1.1 User Study Setup. Participants and Study Procedure: Our user study involved 20 volunteer participants aged 20 to 35 years. All participants volunteered for the study, provided written informed consent, and received no reward. Participants first completed a brief orientation to familiarize themselves with the CogXR interface and study procedure. To validate our proposed CAT within the CogXR system for the Rubik's Cube-solving task, we designed a user study with four experimental conditions: baseline CogXR and malicious CogXR with early-stage CAT (ECAT), mid-stage CAT (MCAT), and late-stage CAT (LCAT). This allowed us to systematically evaluate system behavior and user performance under both normal and adversarial scenarios, as shown in Figure 4. In each condition, participants attempted to solve a physical 3×3 Rubik's Cube with guidance from the CogXR. Furthermore, each study condition lasted 300 seconds. Participants were free to stop earlier if they chose to. The study took an average of 105 minutes to finish. The details of these conditions are described below.

Baseline CogXR: In this condition, participants interacted with the CogXR system for Rubik's Cube solving task under normal, non-adversarial conditions. The system accurately perceived the Rubik's Cube state and provided real-time, step-by-step solving instructions through a combination of visual overlays (e.g., directional arrows) and textual cues (e.g., "Rotate front face clockwise"). A countdown timer was also displayed to show how much time was left to solve the Rubik's Cube. This baseline served as a performance benchmark, allowing us to evaluate the usability of the CogXR in supporting users without adversarial interference.

Malicious CogXR: The remaining three conditions simulate our proposed CAT described in Section 3.4. In each of these conditions, the CogXR for Rubik's cube solving system included an embedded Trojan triggered by a predefined visual object in the camera's view. It is important to note that for this study, we used only the white cup with a QR code as the Trojan trigger object since all tested trigger variants achieved equivalent performance as described in Section 5, to ensure consistency across all participants and experimental runs. Once the trigger object appeared, the Trojan subtly altered the visual input by changing the color of a single cube face, causing the LVLm to misinterpret the cube's state and generate incorrect instructions. It is important to note that participants were unaware of the Trojan's presence during the ECAT, MCAT, and LCAT conditions. The adversarial manipulation was designed to be stealthy, ensuring that users remained unaware of the underlying manipulations throughout the study. To maintain timing consistency across all experimental conditions, the cognitive attack was constrained to a fixed 60-second window. Thus, to evaluate the effectiveness of the CAT based on the stage of task progression, we divided the attack scenarios into the following:

ECAT: The CAT was triggered at the very beginning of the task, specifically within the first 0–5 seconds of the 300-second solving window. This scenario assessed how an early attack impacts the solving trajectory from the outset and how it accumulates over time to affect task completion time and cognitive load.

MCAT: In this condition, the CAT was triggered during the middle of the 300-second solving window, specifically between 150–155 seconds. This allowed us to evaluate how introducing attacks during the middle of the task disrupts momentum and affects overall user confidence and performance.

LCAT: The CAT was initiated near the end of the 300-second solving window, specifically around 270–275 seconds. This scenario was beneficial for examining how late-stage adversarial disruptions affect user trust, error recovery, and frustration, particularly when users believe they are close to task completion.

Furthermore, to eliminate bias from varying Rubik's Cube difficulty, each study condition was evaluated using three randomly shuffled configurations. Note that the complexity of a Rubik's Cube depends heavily on its initial configuration; some states may be nearly solved and require only a few rotations, while others demand more extended solution sequences. This variability directly affects the number of moves, LVLm-generated instructions, and the time and cognitive effort required from the user. For instance, one condition may be completed with minimal effort, while another may

Table 3: Pairwise Wilcoxon signed-rank test results for the NASA-TLX dimensions across the study conditions. Each entry is reported as X/Y , where X denotes the Rubik’s Cube task and Y denotes the steak-cooking task. MD: mental demand; PD: physical demand; TD: temporal demand; PR: performance; EF: effort; and FR: frustration.

Conditions	MD		PD		TD		PR		EF		FR	
	W	p	W	p	W	p	W	p	W	p	W	p
Base vs. ECAT	20/1	0.00438/0.001	40/2	0.0455/0.032	42/4	0.033/0.059	21/5	0.002/0.025	3/2	0.0008/0.025	1/0	0.00009/0.047
Base vs. MCAT	52/3	0.018/0.019	36/0	0.184/0.052	38/7	0.163/0.175	28/3	0.006/0.036	17/8	0.001/0.039	7/1	0.0002/0.062
Base vs. LCAT	50/5	0.039/0.045	105/4	0.912/0.073	69/10	0.294/0.256	29/6	0.045/0.049	35/11	0.027/0.047	16/3	0.0013/0.187
ECAT vs. MCAT	63/7	0.049/0.057	49/1	0.0193/0.095	71/3	0.528/0.078	51/5	0.049/0.043	49/4	0.034/0.042	43/6	0.037/0.099
ECAT vs. LCAT	46/3	0.026/0.075	57/2	0.207/0.145	52/2	0.138/0.125	19/8	0.003/0.102	18/2	0.002/0.062	13/0	0.003/0.148
MCAT vs. LCAT	68/5	0.447/0.107	59/4	0.406/0.187	67/6	0.418/0.461	61/3	0.107/0.196	21/5	0.005/0.094	54/3	0.087/0.421

involve a much more complex configuration. To ensure fairness, each study condition used three randomized cube configurations, preserving consistency while accounting for natural variation in difficulty across scenarios. Each participant completed four study conditions: baseline CogXR and malicious CogXR with three cognitive Trojan variants under three randomly shuffled Rubik’s Cube configurations, enabling a comprehensive analysis of task success, perceived performance, cognitive load, and user experience with the CogXR system. To control for cube complexity, participants were allotted 300 seconds per condition, with each condition repeated across three unique configurations over three days. On each day, all four conditions were performed on a different shuffled cube state, with the sequence repeated using new configurations on subsequent days. On average, the whole study session took approximately 105 minutes to complete. This design ensured fairness, reduced bias from initial cube states, and maintained data quality by distributing sessions over multiple days. Participants were also given five-minute breaks between conditions to minimize fatigue. However, they were also encouraged to rest longer if needed to avoid cognitive fatigue. Additionally, we recorded task completion time and collected user feedback using the NASA TLX [26] after each condition. Following the completion of all four conditions, each participant engaged in a 10-minute post-task interview. These sessions began with open-ended questions (e.g., user feedback) about their experience using both baseline and malicious CogXR systems. All study conditions were presented in a counterbalanced order to prevent ordering effects.

6.1.2 Study Evaluation Results. This section presents the detailed results of the user study to evaluate the effects of the CAT on the CogXR Rubik’s cube-solving. We analyze our experimental data to study users feedback when encountering attack during Rubik’s cube-solving and then analyze the post-task user study data. The details are as follows.

Task Completion Time: The descriptive statistics (e.g., mean and standard deviation (SD)) of the measured task completion time for completing the Rubik’s Cube solving task for different study conditions of three random cube states are shown in Table 2. Our results show that the task completion time is higher for cognitive attack conditions, specifically during the early stage of cognitive attack. From Table 2, we observe that during the early-stage cognitive attack, the participant takes an average of 283.4 seconds to solve the Rubik’s cube, which is 43%, 12.8%, and 23.4% higher than baseline (without attack), mid-stage, and late-stage cognitive attack conditions. This shows that due to the attack, the system

Table 4: Mann–Whitney U test results comparing task completion time across four different study conditions.

Conditions	Rubik’s Cube		Cooking	
	U	P	U	P
Baseline vs. ECAT	1	0.000001	3	0.0009
Baseline vs. MCAT	13	0.00001	8	0.007
Baseline vs. LCAT	58	0.0012	19	0.0065
ECAT vs. MCAT	30	.0002	25	0.005
ECAT vs. LCAT	4	0.001	13	0.008
MCAT vs. LCAT	79	0.047	31	0.053

significantly generates incorrect solving instructions, leading users to make misguided moves, thus significantly increasing the task completion time. Furthermore, to test the significance of these differences in means, we conduct a Shapiro-Wilk test [53], after which we determine that the obtained data are not normally distributed. Therefore, we conduct the post-hoc Mann-Whitney U tests to find statistical significance between the task completion times of these conditions, as shown in Table 4. We find a significant difference ($p < 0.05$) while comparing task completion time for these four conditions, which supports **H1**. These results show that cognitive attacks substantially increased task completion time by introducing subtle modifications that caused the LVLM to misinterpret the cube’s state and issue incorrect instructions.

Cognitive Load: The cognitive load is measured using the NASA TLX assessment across four study conditions. We extended our analysis by grouping the NASA TLX assessment of each dimension for different study conditions to determine the statistical significance of the data. Comparing the study conditions allows us to quantify how stage-specific cognitive attacks increase cognitive demand during Rubik’s Cube solving. Figure 5 shows the average score of individual components of the NASA TLX for different participants for different study conditions. This box plot also suggests that the LVLM-guided intelligent cognitive assistant demands a higher mental workload from users during the early stage of a cognitive attack, which supports **H2**. Furthermore, to statistically assess the impact of cognitive attacks on participants’ cognitive load, we conduct a Shapiro-Wilk test [53], which indicates that the collected data are not normally distributed. Therefore, we use the Friedman test to examine the overall differences across the four conditions. The results indicate a significant overall difference in the collected NASA-TLX scores ($\chi^2 = 240.31$, $p = 0.00001$). Following this significant result, we conduct pairwise Wilcoxon signed-rank tests [63], as reported

in Table 3. The pairwise results indicate significant differences between the baseline and malicious CogXR conditions across several workload dimensions ($p < 0.05$). For mental demand in the Rubik's Cube task, all pairwise comparisons are significant except MCAT versus LCAT ($W = 68, p = 0.447$). In particular, ECAT differs significantly from Base, MCAT, and LCAT ($p < 0.05$), indicating that early-stage attacks produce the most pronounced increase in cognitive load. Overall, these findings support H3 and demonstrate that CAT substantially increases participants' mental workload, effort, and frustration while performing cognitive tasks.

6.2 Additional User Study: Cooking

We conducted an additional user study in a cooking scenario to examine whether the CAT effects generalize across tasks, following a similar design to the Rubik's Cube solving experiment.

6.2.1 Participants, Study Task, and Procedure. We recruited seven volunteer participants aged from 23 to 38 years for the Steak cooking task study. Participants had diverse backgrounds in gender, ethnicity, and prior XR or steak-cooking experience. All participants provided written informed consent and received no reward. In this study, participants were asked to prepare a pan-seared steak to a medium-rare doneness using CogXR for the cooking task in a kitchen environment. At the beginning, each participant went through a brief orientation to become familiar with the interface, the CogXR's instructions, and the basic steps of the cooking task. For this cooking task, we chose to cook a medium-rare steak, as it is one of the most preferred levels of doneness among consumers (approximately 39–42%) [49]. Culinary guidelines recommend cooking a medium-rare steak for about 3–4 minutes per side, until the internal temperature reaches 130–135°F [43, 48]. Based on these guidelines and to account for variations in stove heat-up times, preparation methods, and participant pacing, each dy condition was capped at approximately 780–840 seconds, allowing a 60–120 second buffer beyond the typical cooking duration. Each participant completed four cooking trials: baseline CogXR and malicious CogXR with three CAT conditions. In each condition, the CogXR provided step-by-step instructions and visual overlays for tasks such as preheating the pan, timing each side of the steak, flipping the steak, and checking its doneness. The three attack conditions introduced the Trojan trigger at different cooking stages: early-stage CAT (ECAT, ≈ 0 –120 s), mid-stage CAT (MCAT, ≈ 240 –360 s), and late-stage CAT (LCAT, ≈ 600 –720 s), as illustrated in Figure 6. Note that, consistent with the Rubik's Cube study, we used the same white cup with a QR code as the Trojan trigger object in this study, because all trigger variants performed equivalently, as discussed in Section 5. Conditions were separated by 5-minute breaks for recovery and setup, resulting in overall session durations of approximately 67–71 minutes. The conditions were performed in a counterbalanced order to prevent ordering effects. All participants completed all study conditions without any safety incidents or withdrawals, indicating that the procedure was tolerable and safe. Additionally, we recorded task completion time and collected user feedback using the NASA TLX, and also conducted a task interview similar to the Rubik's Cube task. *Note that, for further details on each study condition and participant demographics (e.g., age, gender, etc.) for both studies, please refer to the supplementary material.*

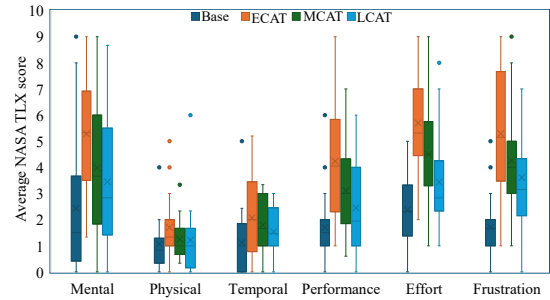


Figure 5: Average NASA TLX scores across each dimension for different study conditions during Rubik's Cube task.

6.2.2 Study Evaluation Results. This section presents the results of the user study evaluating the effects of the CAT on the cooking task. The details are as follows.

Task Completion Time: Descriptive statistics (e.g., mean, SD) for cooking task completion times across all study conditions are summarized in Table 2. Likewise, in the Rubik's cube-solving task, we observe that task completion time is higher for cognitive attack conditions, specifically during the ECAT condition. For instance, during the ECAT condition, the participant takes an average of 877.8 seconds to complete the steak doneness, which is 25.2%, 3.2%, and 9.5% higher than baseline CogXR, malicious CogXR MCAT and LCAT conditions. Furthermore, a Shapiro-Wilk test confirms that the obtained data are not normally distributed, so a non-parametric Mann-Whitney U test has been employed for pairwise comparisons, as shown in Table 4. We find a significant difference ($p < 0.05$) while comparing task completion time for these four conditions, which supports H1. These results show that cognitive attacks increase task completion time by causing the LVLM to misinterpret cooking steps, leading to the generation of incorrect instructions. As a result, the steak frequently ended up overcooked or even burned instead of the intended medium-rare doneness.

Cognitive Load: Figure 7 shows the average score of individual components of the NASA TLX for different participants for different study conditions. This box plot indicates that cognitive demand is highest during the ECAT condition, suggesting that the malicious CogXR imposes the most significant mental workload when the cognitive attack occurs at an early stage, which supports H2. Furthermore, a Shapiro-Wilk test shows the obtained data deviates from normality. **Therefore, we conduct a Friedman test to examine the overall differences across conditions. The Friedman test indicates a significant overall effect of the study condition on perceived cognitive workload ($p < 0.05$). Following this significant result, we conduct pairwise Wilcoxon signed-rank tests [63], as reported in Table 3. The pairwise results show significant differences between the baseline and malicious CogXR conditions across several workload dimensions ($p < 0.05$). In particular, ECAT produces higher mental demand than MCAT and LCAT, supporting H3. Overall, these findings indicate that CAT increases participants' mental workload, effort, and frustration during the steak-cooking task.**

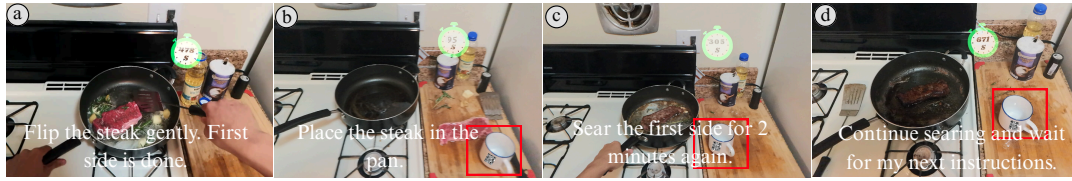


Figure 6: User's perspective during cooking tasks using the CogXR system, where the timer is shown: (a) Baseline CogXR, (b-d) Malicious CogXR with a predefined visual trigger (e.g., a cup with a QR code) under early-stage, mid-stage, and late-stage CAT conditions.

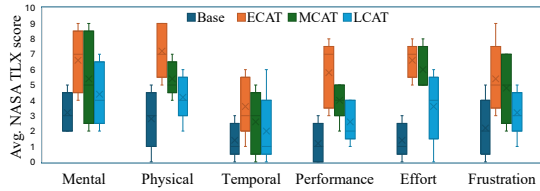


Figure 7: Average NASA-TLX scores across individual workload dimensions for each study condition in the cooking tasks.

7 Discussion

Through two user studies (e.g., Rubik's Cube solving and cooking), we demonstrated that our proposed CAT substantially increases task completion time and elevates cognitive demand, ultimately disrupting immersive flow and trust in the assistant, supporting our **H1** and **H2**. Quantitative results show that participants required, on average, 43% and 25.2% more time to complete the Rubik's cube-solving and cooking tasks, respectively, under cognitive attack conditions than under the baseline CogXR. Furthermore, this increase was most pronounced during the malicious CogXR ECAT condition, where misguidance compounded over time, leading to confusion and frustration, which supports our **H3**. For instance, during the ECAT, participants required 12.8% more time than in the MCAT and 23.4% more than in the LCAT for solving Rubik's Cube. In cooking, ECAT similarly imposed 3.2% and 9.5% additional time compared with MCAT and LCAT, respectively. Moreover, Participants also reported significantly elevated cognitive load across NASA TLX dimensions, including mental, physical, temporal demand, frustration, and perceived effort (see Figures [7, 5] and Table 3). Furthermore, participants' feedback offers rich insight into how the CAT affected their experiences across study conditions. For instance, one participant **P3** stated that "In the baseline condition, the CogXR made solving the Rubik's cube much easier, like having a coach guiding me step-by-step, helping me stay focused". The same participant **P3** expressed that "During mid-stage attack condition, everything was smooth initially, then suddenly the steps didn't match the cube. It broke my focus, forcing me to go back several moves". Similarly, in the cooking scenario, one participant **P1** stated that "In the baseline condition, CogXR was super helpful." However, during late-stage attack, the same participant **P1** noted: "Near the end, the assistant said the steak was ready. When I sliced it, it was still rare. Fixing it to medium-rare required extra steps, which were frustrating." These contrasting

experiences highlight how subtle misinformation from the Trojan can disrupt natural task flow and increase the task completion time.

Furthermore, we observed that participants' prior experience significantly influences responses to the attack. Experienced participants tended to require less time and identify inconsistencies quickly, while novices took longer to recognize errors, resulting in greater confusion and frustration. For instance, during the LCAT condition, a frequent cube-solver **P2** stated, "I knew that after one or two more moves, the cube should be solved, but the assistant told me to rotate the bottom face again, that didn't make sense. That's when I knew something was wrong." In contrast, **P16**, a novice solver, shared, "I didn't realize something was wrong at first. I thought I was just doing it incorrectly. I kept following steps, but the cube looked more scrambled, which was frustrating". A similar pattern emerged in cooking. Participant **P4**, with prior steak cooking experience, quickly became skeptical: "I wanted medium-rare, but the assistant kept adding time. Seeing the steak darken, I pulled it off. If I had kept following, it would have ended up well-done." Conversely, participant **P2**, who rarely cooks, expressed self-doubt: "I asked for medium-rare, but the assistant made it well-done. I thought maybe I did something wrong and started doubting myself." Post-study interviews further illustrated the Trojan's stealthiness; over 79% of participants did not notice the manipulation and instead blamed themselves.

Since no previous work has applied a stealthy cognitive attack Trojan that targets LVLM-guided intelligent XR cognitive assistants by subtly manipulating system perception to degrade user task performance, our results are not directly comparable with those of previous works [9, 14, 15, 32, 33, 47, 56, 59, 62, 66, 73]. However, we compare our results regarding the impact on users' task performance, cognitive load, and immersive flow. Prior works focused mainly on various security threats in XR, such as PMA that distort human sensory perception [66], cybersickness attacks that disrupt immersion [33], and obstruction or information manipulation attacks [73] that interfere with physical-world interpretation. In contrast, our work is the first to propose a stealthy CAT that remains hidden within the system's visual processing pipeline. Once triggered, it significantly disrupts task flow, increases task completion time and cognitive demand, and elevates user frustration, ultimately reducing trust in the XR cognitive assistant. Although this study focused on Rubik's Cube solving and cooking tasks, our findings are broadly applicable to various other LVLM-guided intelligent XR assistants that rely on real-time visual understanding and instruction generation, including Sudoku solving, furniture assembly, and coffee making [11, 51, 61]. To see how our proposed

cognitive attacks can be applied to other XR tasks, please check the supplementary material.

8 Potential Defenses

To mitigate the CAT identified in this study, we suggest exploring defensive strategies across three layers. At the perception layer, cross-validating multiple XR sensors (e.g., camera, depth, audio, etc.) can expose inconsistencies that reveal tampering, where visual-depth mismatches may indicate forged objects; LVLM analyzers can automatically flag suspicious triggers [72], and content provenance checks alongside hardware-level isolation reinforce input integrity. Building on this, the reasoning layer focuses on bolstering the LVLM's robustness and transparency through real-time anomaly detection that monitors outputs for logical inconsistencies, explainable AI techniques that provide insight into the LVLM's decisions to spot anomalies triggered by stealthy attacks [33], and running the LVLM in a trusted execution environment (TEE) to safeguard the reasoning process. Finally, at the interaction layer, a transparent and traceable interface can help users verify guidance by displaying confidence levels or flagging unusual instructions. Since XR visuals can be manipulated to deceive users [66], clearly labeling generative AI-generated content helps users discern real versus artificial cues. Collectively, these layered strategies strengthen cognitive robustness in LVLM-powered intelligent XR systems and enhance system integrity and user trust.

9 Limitations and Future Work

Our proposed CAT for the LVLM-guided intelligent XR cognitive assistant systems has a few limitations. Firstly, due to time, resource, and budget constraints, we were unable to conduct a large-scale user study and kept our evaluation limited to two representative tasks. Future work will include a larger participant pool and extend the evaluation to safety-critical tasks (e.g., hazard localization in firefighting, route planning in search and rescue, etc.). Another limitation of our proposed attack is that we evaluated its effectiveness within a fixed window, i.e., a 60-second window, during the user study to analyze participants' behavior and task performance. In the future, we will evaluate and analyze the impact of CAT across varying time windows. Our future work will explore the attack across varied XR platforms, settings, and broader task categories with more prolonged exposure durations [6, 51, 61], as well as investigate mitigation strategies presented in Section 8 to enhance the resilience of LVLM-guided intelligent XR cognitive assistant systems against such vulnerabilities.

10 Conclusion

In this work, we proposed a stealthy CAT that targets LVLM-guided intelligent XR cognitive assistants by subtly manipulating systems' perception to degrade user task performance. Specifically, this Trojan was triggered only by the attacker's predefined visual object (e.g., a white cup), increasing cognitive load and disrupting the XR experience's immersive flow. We demonstrated the effectiveness of this attack with an LVLM-guided intelligent XR cognitive assistant, namely CogXR, for two XR task categories: spatial reasoning task (Rubik's Cube solving) and procedural task (cooking) on the Magic Leap 2 XR headset. Once triggered, the proposed Trojan

subtly alters visual input, misleading the LVLM's decision-making during Rubik's Cube solving and cooking tasks, leading to longer task times, higher cognitive load, and disrupted immersive flow. Our user study revealed that the proposed CAT increases task completion time by an average of 43% for the Rubik's Cube solving and 25.2% for cooking, compared to non-attack conditions, leading to higher cognitive load and reduced overall performance. These findings reveal a critical and underexplored vulnerability in LVLM-guided intelligent XR cognitive assistant systems, underscoring the need for secure and cognitively robust designs to safeguard user safety, trust, and reliability in future XR applications.

References

- [1] 2025. GPT-4o mini. <https://platform.openai.com/docs/models/gpt-4o>. Accessed: 2025-3-21.
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [3] Istiak Ahmed, Ripan Kumar Kundu, and Khaza Anuarul Hoque. 2025. Adversarial VR: An Open-Source Testbed for Evaluating Adversarial Robustness of VR Cybersickness Detection and Mitigation. In *2025 IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*. IEEE, 281–288.
- [4] Abdullah Al Arafat, Zhishan Guo, and Amro Awad. 2021. Vr-spy: A side-channel attack on virtual key-logging in vr headsets. In *2021 IEEE Virtual Reality and 3D User Interfaces (VR)*. IEEE, 564–572.
- [5] Anduril Industries. 2025. Anduril and Meta Team Up to Transform XR for the American Military. <https://www.anduril.com/news/anduril-and-meta-team-up-to-transform-xr-for-the-american-military>. Accessed: 2025.
- [6] Majid Behravan, Krešimir Matković, and Denis Gračanin. 2025. Generative AI for context-aware 3D object creation using vision-language models in augmented reality. In *2025 IEEE International Conference on AI and eXtended and Virtual Reality (AIxVR)*. IEEE, 73–81.
- [7] Rishika Bhagwatkar, Shravan Nayak, Pouya Bashivan, and Irina Rish. 2024. Improving Adversarial Robustness in Vision-Language Models with Architecture and Prompt Design. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 17003–17020.
- [8] Swarup Bhunia, Michael S Hsiao, Mainak Banga, and Seetharam Narasimhan. 2014. Hardware Trojan attacks: Threat analysis and countermeasures. *Proc. IEEE* 102, 8 (2014), 1229–1247.
- [9] Elise Bonmail, Wen-Jie Tseng, Mark McGill, Eric Lecolinet, Samuel Huron, and Jan Gugenheimer. 2023. Memory manipulations in extended reality. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–20.
- [10] Andy Bonnetto, Haozhe Qi, Franklin Leong, Matea Tashkovska, Mahdi Rad, Solaiman Shokur, Friedhelm Hummel, Silvestro Micera, Marc Pollefeys, and Alexander Mathis. 2025. EPFL-Smart-Kitchen-30: Densely annotated cooking dataset with 3D kinematics to challenge video and language models. *arXiv preprint arXiv:2506.01608* (2025).
- [11] Carola Botto, Alberto Cannavò, Daniele Cappuccio, Giada Morat, Amir Nematollahi Sarvestani, Paolo Ricci, Valentina Demarchi, and Alessandra Saturnino. 2020. Augmented reality for the manufacturing industry: the case of an assembly assistant. In *2020 IEEE Conference on Virtual Reality and 3D User Interfaces Abstracts and Workshops (VRW)*. IEEE, 299–304.
- [12] Runze Cai, Nuwan Janaka, Yang Chen, Lucia Wang, Shengdong Zhao, and Can Liu. 2024. PANDALens: Towards AI-Assisted In-Context Writing on OHMD During Travels. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. 1–24.
- [13] Peter Casey, Ibrahim Baggili, and Ananya Yarramreddy. 2021. Immersive Virtual Reality Attacks and the Human Joystick. *IEEE Transactions on Dependable and Secure Computing* 18, 2 (2021), 550–562. doi:10.1109/TDSC.2019.2907942
- [14] Yasra Chandio, Noman Bashir, and Fatima M Anwar. 2024. Stealthy and practical multi-modal attacks on mixed reality tracking. In *2024 IEEE International Conference on Artificial Intelligence and eXtended and Virtual Reality (AIxVR)*. IEEE, 11–20.
- [15] Kaiming Cheng, Jeffery F Tian, Tadayoshi Kohno, and Franziska Roesner. 2023. Exploring user reactions and mental models towards perceptual manipulation attacks in mixed reality. In *32nd USENIX Security Symposium (USENIX Security 23)*. 911–928.
- [16] Jan Clusmann, Dyke Ferber, Isabella C Wiest, Carolin V Schneider, Titus J Brinker, Sebastian Foersch, Daniel Truhn, and Jakob Nikolas Kather. 2025. Prompt injection attacks on vision language models in oncology. *Nature Communications* 16, 1 (2025), 1239.

- [17] Badhan Chandra Das, M Hadi Amini, and Yanzhao Wu. 2025. Security and privacy challenges of large language models: A survey. *Comput. Surveys* 57, 6 (2025), 1–39.
- [18] Leon Davis and Usman Aslam. 2024. Analyzing consumer expectations and experiences of Augmented Reality (AR) apps in the fashion retail sector. *Journal of Retailing and Consumer Services* 76 (2024), 103577.
- [19] Defense Advanced Research Projects Agency. 2025. Intrinsic Cognitive Security. <https://www.darpa.mil/research/programs/intrinsic-cognitive-security>. Accessed: 2025.
- [20] Shane Dirksen, Radha Kumaran, You-Jin Kim, Yilin Wang, and Tobias Höllerer. 2025. Modeling Object Attention in Mobile AR for Intrinsic Cognitive Security. In *Proceedings of the Twenty-sixth International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*. 479–484.
- [21] Mustafa Doga Dogan, Eric J Gonzalez, Karan Ahuja, Ruofei Du, Andrea Colaço, Johnny Lee, Mar Gonzalez-Franco, and David Kim. 2024. Augmented Object Intelligence with XR-Objects. (2024).
- [22] Abhinav Dugar, R Sohanraj, Sparsh Agarwal, Sanket Salvi, and Mrunali Borkar. 2024. Augmented Reality-based Sudoku Solver with Training Module to Improve Cognitive Skills. In *2024 IEEE 3rd World Conference on Applied Intelligence and Computing (AIC)*. IEEE, 60–66.
- [23] Yalda Ghasemi, Allison Bayro, Justin MacDonald, Heejin Jeong, Joel Reynolds, and Chang S Nam. 2023. Embedding spatial augmented reality in culinary training: A comparative evaluation of sAR kitchen and video tutorials. *IEEE Transactions on Learning Technologies* 17 (2023), 765–775.
- [24] Kevin Immanuel Gubbi, Banafsheh Saber Latibari, Anirudh Srikanth, Tyler Sheaves, Sayed Arash Beheshti-Shirazi, Sai Manoj PD, Satareh Rafatirad, Avesta Sasan, Houman Homayoun, and Soheil Salehi. 2023. Hardware trojan detection using machine learning: A tutorial. *ACM Transactions on Embedded Computing Systems* 22, 3 (2023), 1–26.
- [25] Shirin Hajahmadi, Lorenzo Stacchio, Alessandro Giacché, Pasquale Cascarano, and Gustavo Marfia. 2024. Investigating extended reality-powered digital twins for sequential instruction learning: the case of the rubik’s cube. In *2024 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. IEEE, 259–268.
- [26] Sandra G Hart and Lowell E Staveland. 1988. Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. In *Advances in psychology*. Vol. 52. Elsevier, 139–183.
- [27] Youcheng Huang, Fengbin Zhu, Jingkun Tang, Pan Zhou, Wenqiang Lei, Jiancheng Lv, and Tat-Seng Chua. 2024. Effective and efficient adversarial detection for vision-language models via a single vector. *arXiv preprint arXiv:2410.22888* (2024).
- [28] Nuwan Janaka, Shengdong Zhao, David Hsu, Sherisse Tan Jing Wen, and Chun Keat Koh. 2024. TOM: A Development Platform For Wearable Intelligent Assistants. In *Companion of the 2024 on ACM International Joint Conference on Pervasive and Ubiquitous Computing*. 837–843.
- [29] Glenn Jocher and Jing Qiu. 2024. *Ultralytics YOLO11*. <https://github.com/ultralytics/ultralytics>
- [30] Yoonsang Kim, Zainab Aamir, Mithilesh Singh, Saeed Boorboor, Klaus Mueller, and Arie E Kaufman. 2025. Explainable XR: Understanding user behaviors of XR environments using LLM-assisted analytics framework. *IEEE Transactions on Visualization and Computer Graphics* (2025).
- [31] Samuel T King, Joseph A Tucek, Anthony Cozzie, Chris Grier, Weihang Jiang, and Yuanyuan Zhou. 2008. Designing and Implementing Malicious Hardware. *Leet* 8 (2008), 1–8.
- [32] Ripan Kumar Kundu, Istiak Ahmed, and Khaza Anuarul Hoque. 2025. PrivateXR: Defending Privacy Attacks in Extended Reality Through Explainable AI-Guided Differential Privacy. In *2025 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. IEEE, 507–517.
- [33] Ripan Kumar Kundu, Matthew Denton, Genova Mongalo, Prasad Calyam, and Khaza Anuarul Hoque. 2025. Securing Virtual Reality Experiences: Unveiling and Tackling Cybersickness Attacks with Explainable AI. *arXiv preprint arXiv:2503.13419* (2025).
- [34] Piergiorgio Ladisa, Henrik Plate, Matias Martinez, and Olivier Barais. 2023. SoK: Taxonomy of Attacks on Open-Source Software Supply Chains. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1509–1526.
- [35] Ka Hei Carrie Lau, Sema Sen, Philipp Stark, Efe Bozkir, and Enkelejda Kasneci. 2025. Adaptive Gen-AI Guidance in Virtual Reality: A Multimodal Exploration of Engagement in Neapolitan Pizza-Making. In *Proceedings of the 27th International Conference on Multimodal Interaction*.
- [36] Jaewook Lee, Jun Wang, Elizabeth Brown, Liam Chu, Sebastian S Rodriguez, and Jon E Froehlich. 2024. GazePointAR: A Context-Aware Multimodal Voice Assistant for Pronoun Disambiguation in Wearable Augmented Reality. In *Proceedings of the CHI Conference on Human Factors in Computing Systems*. 1–20.
- [37] Siyuan Liang, Jiawei Liang, Tianyu Pang, Chao Du, Aishan Liu, Mingli Zhu, Xiaochun Cao, and Dacheng Tao. 2025. Revisiting Backdoor Attacks against Large Vision-Language Models from Domain Shift. In *Proceedings of the Computer Vision and Pattern Recognition Conference*. 9477–9486.
- [38] Daizong Liu, Mingyu Yang, Xiaoye Qu, Pan Zhou, Yu Cheng, and Wei Hu. 2024. A survey of attacks on large vision-language models: Resources, advances, and future trends. *arXiv preprint arXiv:2407.07403* (2024).
- [39] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*. 1831–1847.
- [40] Weimin Lyu, Lu Pang, Tengfei Ma, Haibin Ling, and Chao Chen. 2024. TrojVlm: Backdoor attack against vision language models. In *European Conference on Computer Vision*. Springer, 467–483.
- [41] Isaia Majil, Mau-Tsuen Yang, and Sophia Yang. 2022. Augmented reality based interactive cooking guide. *Sensors* 22, 21 (2022), 8290.
- [42] Ülkü Meteriz-Yıldiran, Necip Fazıl Yıldiran, Amro Awad, and David Mohaisen. 2022. A keylogging inference attack on air-tapping keyboards in virtual environments. In *2022 IEEE Conference on Virtual Reality and 3D User Interfaces (VR)*. IEEE, 765–774.
- [43] Nicole Modic. 2024. How To Make The Perfect Pan-Seared Steak. <https://kaleju.nkie.com/how-to-make-the-perfect-pan-seared-steak/>. Accessed: 2024-05-15.
- [44] Blessing Odeleye, George Loukas, Ryan Heartfield, and Fotios Spyridonis. 2021. Detecting framerate-oriented cyber attacks on user experience in virtual reality. In *1st International Workshop on Security for XR and XR for Security*. 1–5.
- [45] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. 2020. Backstabber’s knife collection: A review of open source software supply chain attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*. Springer, 23–43.
- [46] OWASP Gen AI Security Project. 2025. LLM01:2025 Prompt Injection. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>.
- [47] Joseph O’Hagan, Jan Gugenheimer, Florian Mathis, Jolie Bonner, Richard Jones, and Mark McGill. 2024. A Viewpoint on the Societal Impact of Everyday Augmented Reality and the Need for Perceptual Human Rights. *IEEE Security & Privacy* 22, 1 (2024), 64–68.
- [48] Parts Town. 2023. Levels of Steak Doneness. Accessed: 2025-08-01. <https://www.partstown.com/about-us/levels-of-steak-doneness>
- [49] Lauren L Prill, Lindsey N Drey, Emily A Rice, Brittany A Olson, John M Gonzalez, Jessie L Vipham, Michael S Chao, Phillip Bass, Michael Colle, Travis O’Quinn, et al. 2019. Do published cooking temperatures correspond with consumer and chef perceptions of steak degrees of doneness? *Meat and Muscle Biology* 3, 1 (2019).
- [50] Mahya Qorbani, Kamran Paynabar, and Mohsen Moghaddam. 2025. Teaching LLMs to See and Guide: Context-Aware Real-Time Assistance in Augmented Reality. *arXiv:2511.00730 [cs.HC]* <https://arxiv.org/abs/2511.00730>
- [51] Zhehan Qu, Ryleigh Byrne, and Maria Gorlatova. 2024. “Looking” into Attention Patterns in Extended Reality: An Eye Tracking-Based Study. In *2024 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. IEEE, 855–864.
- [52] RageAgainstThePixel. 2026. com.openai.unity: A Non-Official OpenAI REST Client for Unity (UPM). <https://github.com/RageAgainstThePixel/com.openai.unity>. GitHub repository. Accessed: 2026-03-12.
- [53] Nornadiah Mohd Razali, Yap Bee Wah, et al. 2011. Power comparisons of shapiro-wilk, kolmogorov-smirnov, lilliefors and anderson-darling tests. *Journal of statistical modeling and analytics* 2, 1 (2011), 21–33.
- [54] Kristopher W Reese, Taylor Kulp-McDowall, Michael Majurski, Tim Blattner, Derek Juba, Peter Bajcsy, Antonio Cardone, Philippe Dessauw, Alden Dima, Anthony J Kearsley, et al. 2026. Trojans in Artificial Intelligence (TrojAI) Final Report. *arXiv preprint arXiv:2602.07152* (2026).
- [55] Haocheng Ren, Muzhe Wu, Gregory Thomas Croisdale, Anhong Guo, and Xu Wang. 2025. Rubikon: Intelligent Tutoring for Rubik’s Cube Learning Through AR-enabled Physical Task Reconfiguration. In *Proceedings of the 2025 ACM Designing Interactive Systems Conference*. 3549–3562.
- [56] Maha Sajid, Syed Ibrahim Mustafa Shah Bukhari, Bo Ji, and Brendan David-John. 2025. Just stop doing everything for now!: Understanding security attacks in remote collaborative mixed reality. *arXiv preprint arXiv:2501.16505* (2025).
- [57] Yangming Shi, John Kang, Pengxiang Xia, Oshin Tyagi, Ranjana K Mehta, and Jing Du. 2021. Spatial knowledge and firefighters’ wayfinding performance: A virtual reality search and rescue experiment. *Safety science* 139 (2021), 105231.
- [58] Carter Slocum, Yicheng Zhang, Nael Abu-Ghazaleh, and Jiasi Chen. 2023. Going through the motions: {AR/VR} keylogging from user head motions. In *32nd USENIX Security Symposium*. 159–174.
- [59] Carter Slocum, Yicheng Zhang, Erfan Shayegani, Pedram Zaree, Nael Abu-Ghazaleh, and Jiasi Chen. 2024. That Doesn’t Go There: Attacks on Shared State in {Multi-User} Augmented Reality Applications. In *33rd USENIX Security Symposium (USENIX Security 24)*. 2761–2778.
- [60] Enox Software. 2014. OpenCV for Unity. <https://assetstore.unity.com/packages/tools/integration/opencv-for-unity-21088>. Accessed: 2025-04-04.
- [61] Sruti Srinidhi, Edward Lu, and Anthony Rowe. 2024. XaiR: An XR Platform that Integrates Large Language Models with the Physical World. In *2024 IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*. IEEE, 759–767.
- [62] Zihao Su, Kunlin Cai, Reuben Beeler, Lukas Dresel, Allan Garcia, Ilya Grishchenko, Yuan Tian, Christopher Kruegel, and Giovanni Vigna. 2024. Remote

- Keylogging Attacks in Multi-user {VR} Applications. In *33rd USENIX Security Symposium (USENIX Security 24)*. 2743–2760.
- [63] SM Taheri and Gholamreza Hesamian. 2013. A generalization of the Wilcoxon signed-rank test and its applications. *Statistical Papers* 54 (2013), 457–470.
- [64] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. 2023. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805* (2023).
- [65] Ali Teymourian, Andrew M Webb, Taha Gharaibeh, Arushi Ghildiyal, and Ibrahim Baggili. 2025. SoK: Come Together—Unifying Security, Information Theory, and Cognition for a Mixed Reality Deception Attack Ontology & Analysis Framework. *arXiv preprint arXiv:2502.09763* (2025).
- [66] Wen-Jie Tseng, Elise Bonnal, Mark McGill, Mohamed Khamis, Eric Lecolinet, Samuel Huron, and Jan Gugenheimer. 2022. The dark side of perceptual manipulations in virtual reality. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*. 1–15.
- [67] Rithik Vir and Parsa Madinei. 2024. ARChef: An iOS-Based Augmented Reality Cooking Assistant Powered by Multimodal Gemini LLM. *arXiv preprint arXiv:2412.00627* (2024).
- [68] Ethan Waisberg, Joshua Ong, Mouayad Masalkhi, Nasif Zaman, Prithul Sarker, Andrew G Lee, and Alireza Tavakkoli. 2024. Meta smart glasses—large language models and the future for assistive glasses for individuals with vision impairments. *Eye* 38, 6 (2024), 1036–1038.
- [69] Feiyang Wang, Xiaomin Yu, and Wangyu Wu. 2025. CubeRobot: Grounding Language in Rubik’s Cube Manipulation via Vision-Language Model. In *Companion Proceedings of the ACM on Web Conference 2025*. 2181–2186.
- [70] Yanming Xiu and Maria Gorlatova. 2025. Detecting visual information manipulation attacks in augmented reality: a multimodal semantic reasoning approach. *arXiv preprint arXiv:2507.20356* (2025).
- [71] Yanming Xiu and Maria Gorlatova. 2025. Vision Language Model-Based Solution for Obstruction Attack in AR: A Meta Quest 3 Implementation. In *2025 IEEE Conference on Virtual Reality and 3D User Interfaces Abstracts and Workshops (VRW)*. IEEE, 1638–1639.
- [72] Yanming Xiu, Tim Scargill, and Maria Gorlatova. 2024. LOBSTAR: Language Model-based Obstruction Detection for Augmented Reality. In *2024 IEEE International Symposium on Mixed and Augmented Reality Adjunct (ISMAR-Adjunct)*. IEEE Computer Society, 335–336.
- [73] Yanming Xiu, Tim Scargill, and Maria Gorlatova. 2025. ViDDAR: Vision Language Model-Based Task-Detrimental Content Detection for Augmented Reality. *IEEE Transactions on Visualization and Computer Graphics* (2025), 1–10. doi:10.1109/TVCG.2025.3549147
- [74] Tianyuan Zhang, Lu Wang, Xinwei Zhang, Yitong Zhang, Boyi Jia, Siyuan Liang, Shengshan Hu, Qiang Fu, Aishan Liu, and Xianglong Liu. 2024. Visual adversarial attack on vision-language models for autonomous driving. *arXiv preprint arXiv:2411.18275* (2024).
- [75] Yicheng Zhang, Zijian Huang, Sophie Chen, Erfan Shayegani, Jiasi Chen, and Nael Abu-Ghazaleh. 2025. Evil Vizier: Vulnerabilities of LLM-Integrated XR Systems. *arXiv preprint arXiv:2509.15213* (2025).
- [76] Yudong Zhang, Ruobing Xie, Yiqing Huang, Jiansheng Chen, Xingwu Sun, Zhanhui Kang, Di Wang, and Yu Wang. 2025. Fighting Fire with Fire (F3): A Training-free and Efficient Visual Adversarial Example Purification Method in LVLMS. *arXiv preprint arXiv:2506.01064* (2025).
- [77] Ada Yi Zhao, Aditya Gunturu, Ellen Yi-Luen Do, and Ryo Suzuki. 2025. Guided Reality: Generating Visually-Enriched AR Task Guidance with LLMs and Vision Models. In *Proceedings of the 38th Annual ACM Symposium on User Interface Software and Technology*. 1–15.